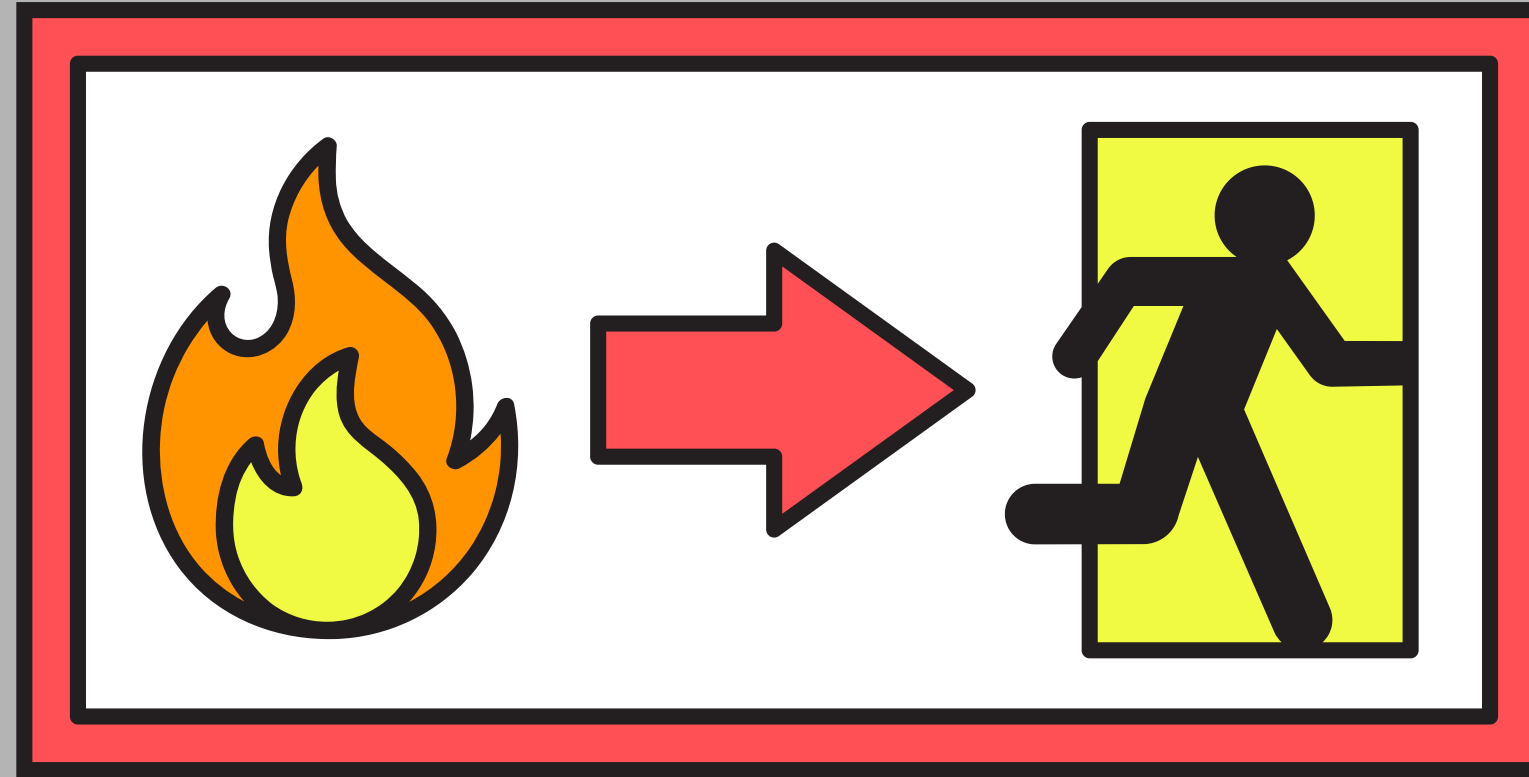


Revisión intermedia

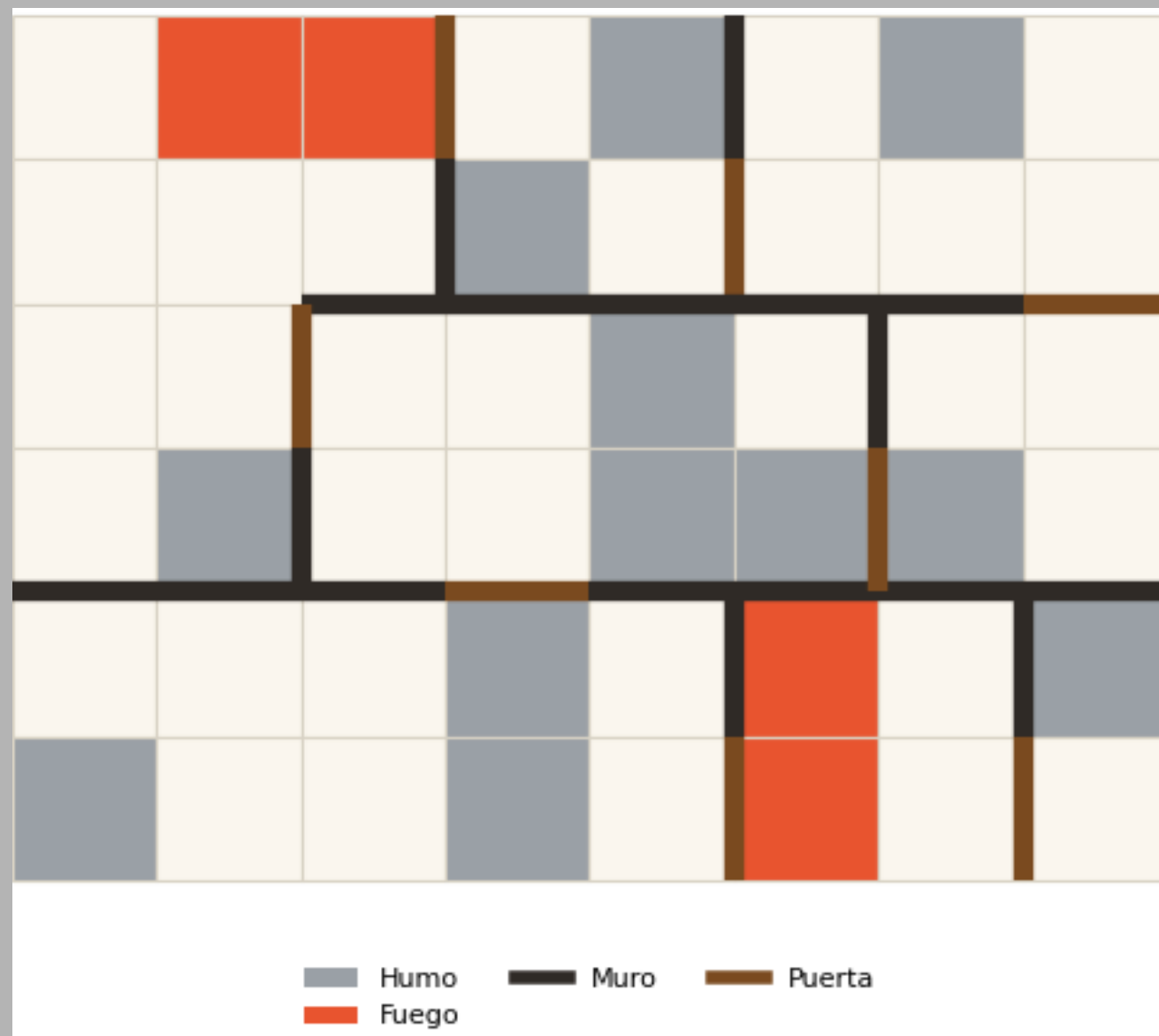


Jesús Osvaldo Ramos Pérez A01713833

Santiago Martín del Campo Soler A01713396

Iker Rodríguez Amaro A01712814

Diseño de interfaz

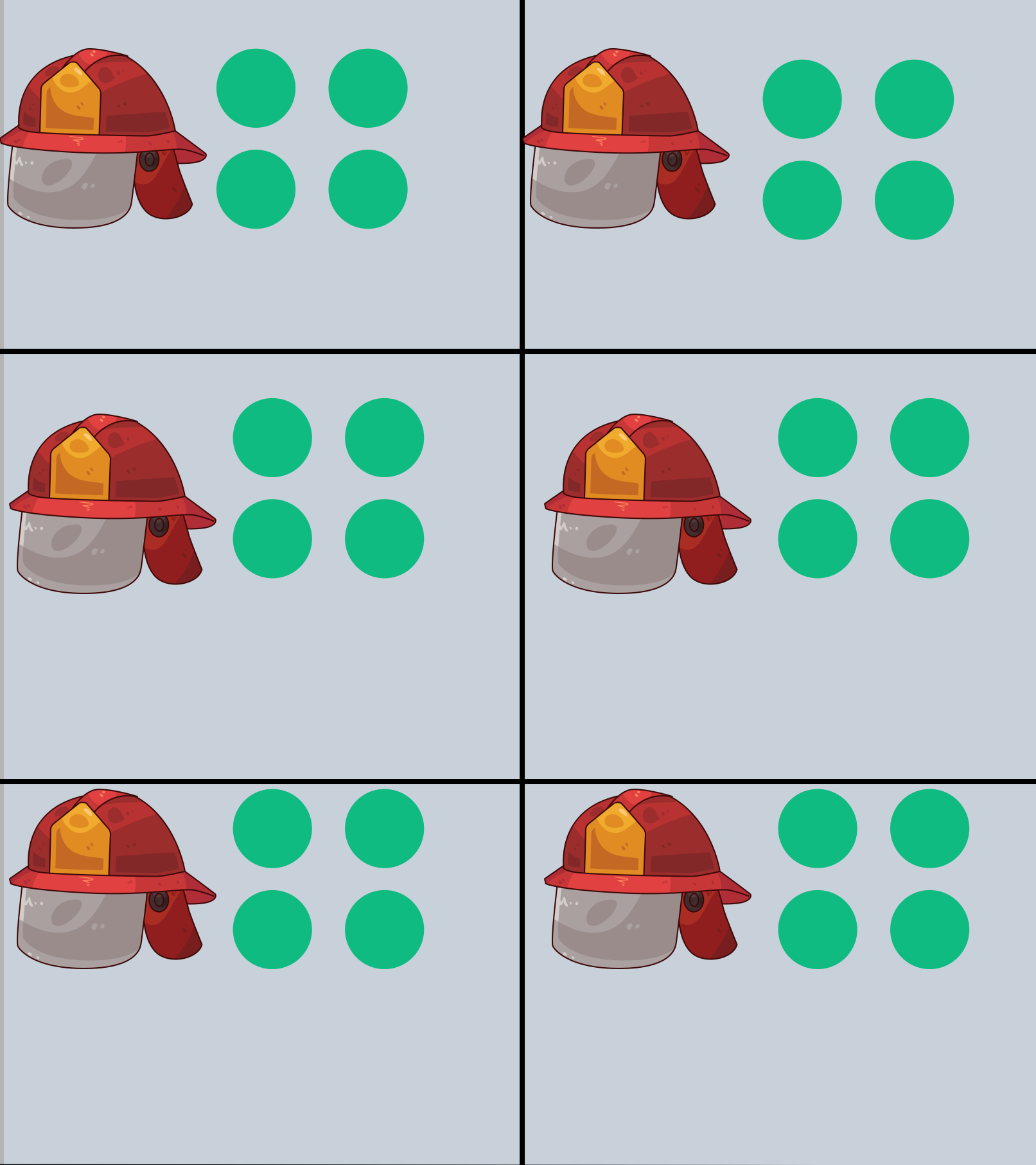


Tablero:

- Bombero
- Fuego
- Humo
- Víctimas
- Muros
- Puertas

UI:

- Víctimas rescatadas
- Víctimas perdidas
- Daño acumulado
- Turno actual
- Estado de cada agente
- Controles de la simulación



Víctimas rescatadas: 2 / 7
Daño estructural: 6 / 25

Víctimas perdidas: 0
Turno: 10

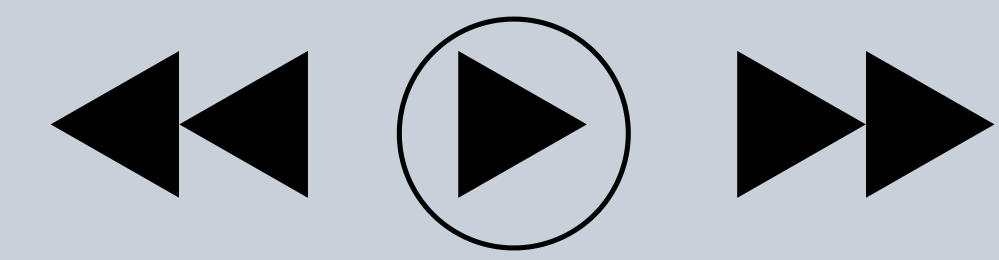


DIAGRAMA DE ESTADOS DE VIDEOJUEGO

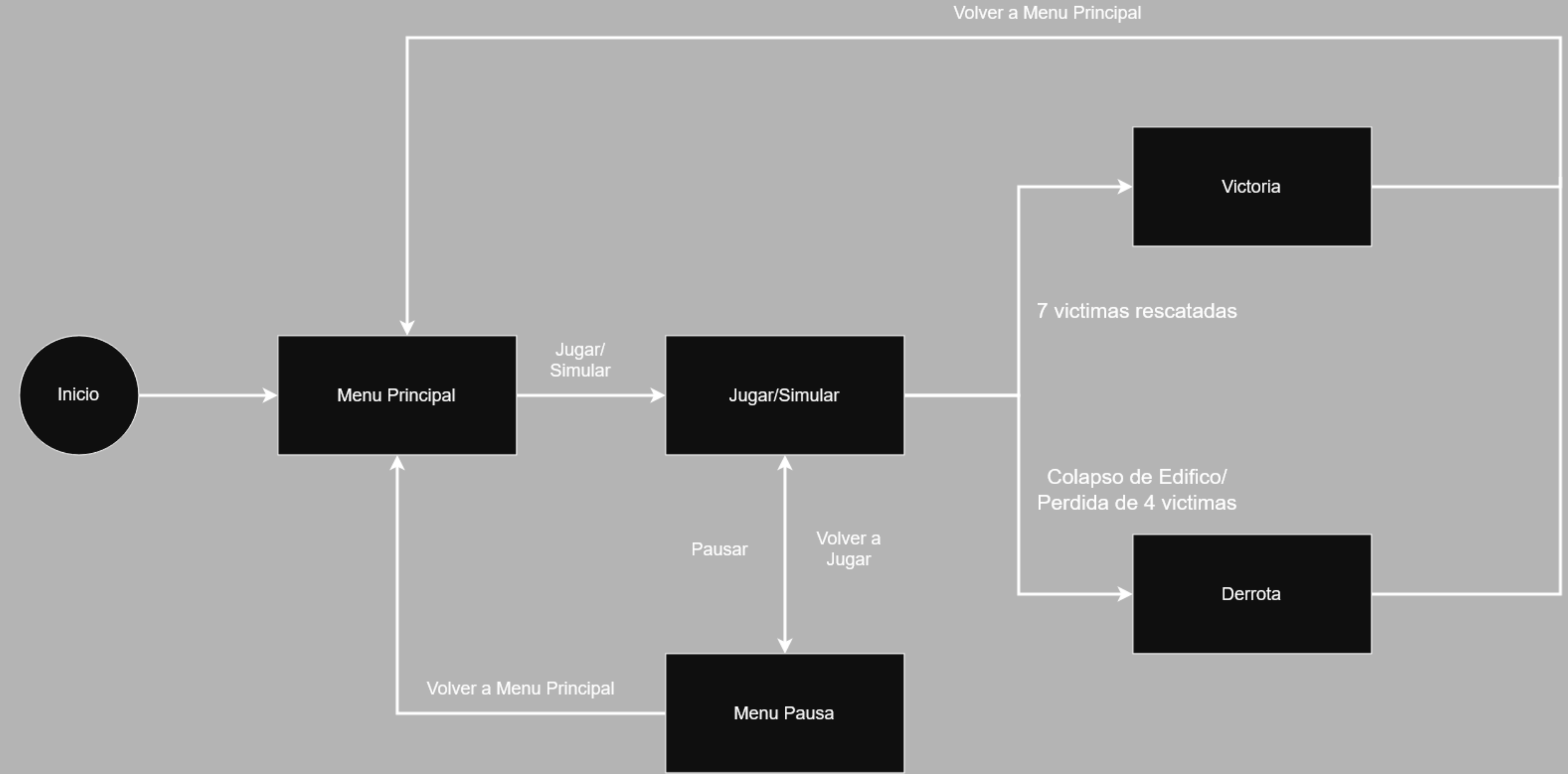


DIAGRAMA DE ESTADOS DE MAPA

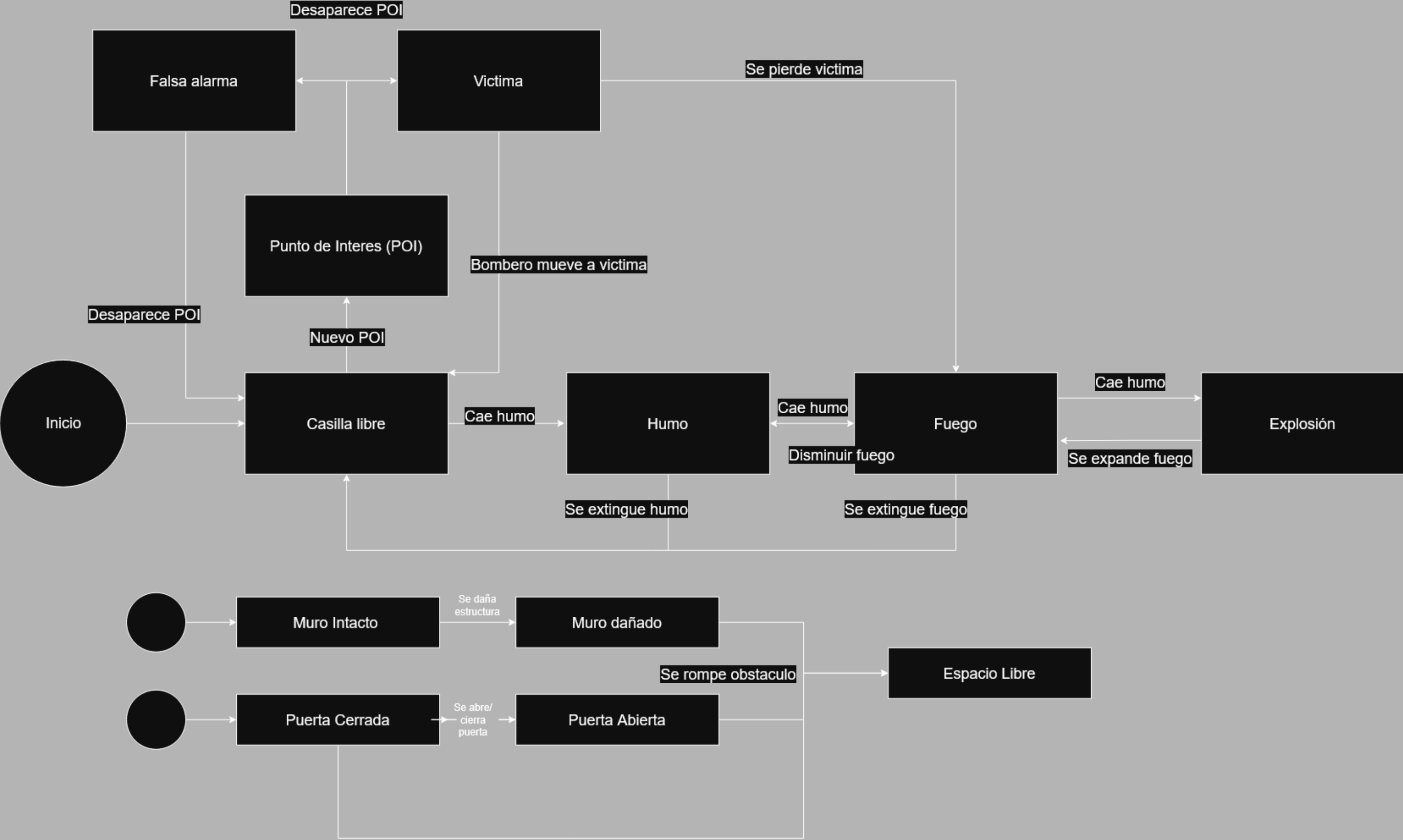
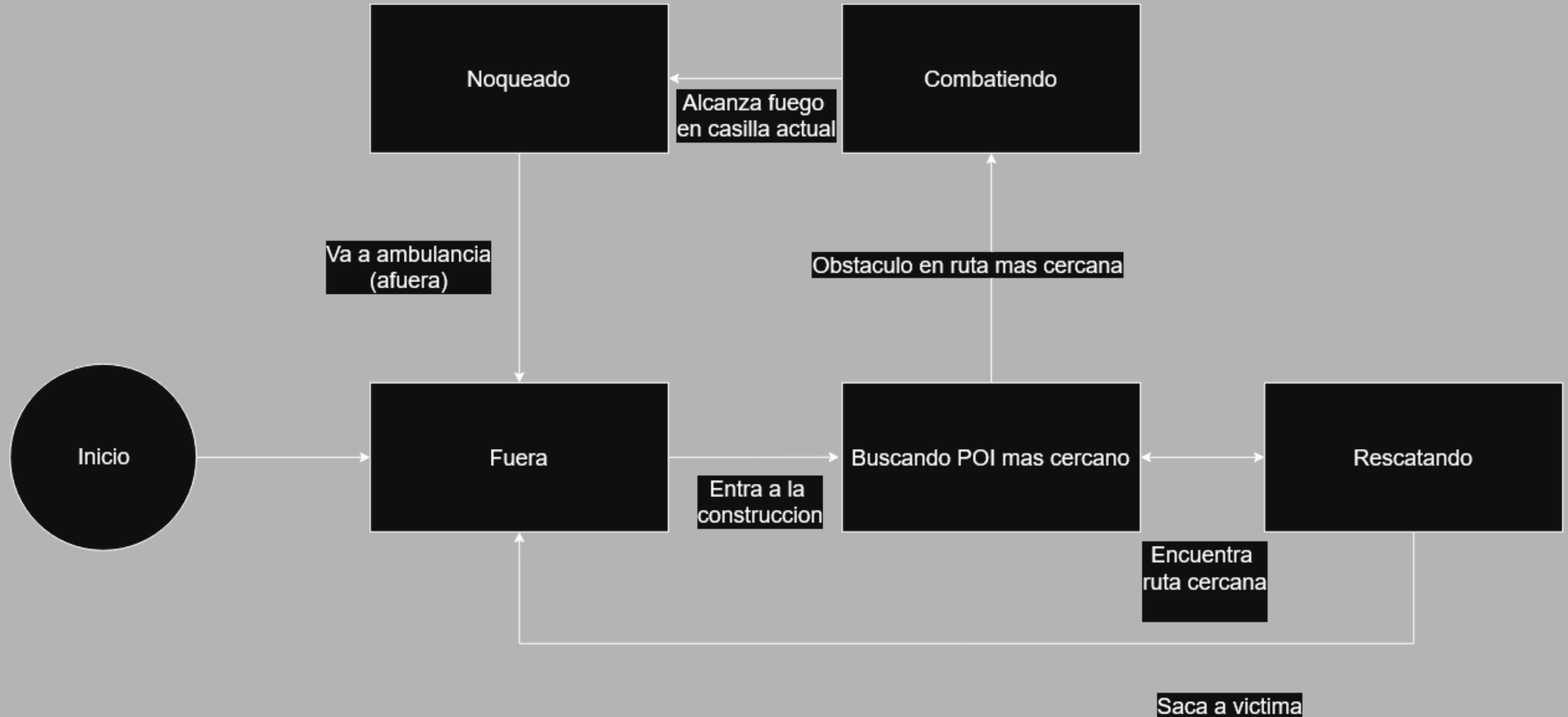


DIAGRAMA DE ESTADOS DE AGENTE



Comportamientos de los agentes

Comportamiento reactivo

El agente observará continuamente el estado de las casillas cercanas. Así el agente podrá percibir cambios en el entorno y responder a ellos.

El agente podría reaccionar ante:

Aperción de fuego, humo, explosiones, bloqueo de una ruta o ante alguna víctima que se encuentre en peligro.



Comportamientos de los agentes

Comportamiento proactivo

El agente no necesita esperar a que algo ocurra para actuar, su meta es localizar y rescatar víctimas antes de que el edificio colapse.

Comportamiento basado en objetivos

Meta global: Rescatar víctimas

Subobjetivos:

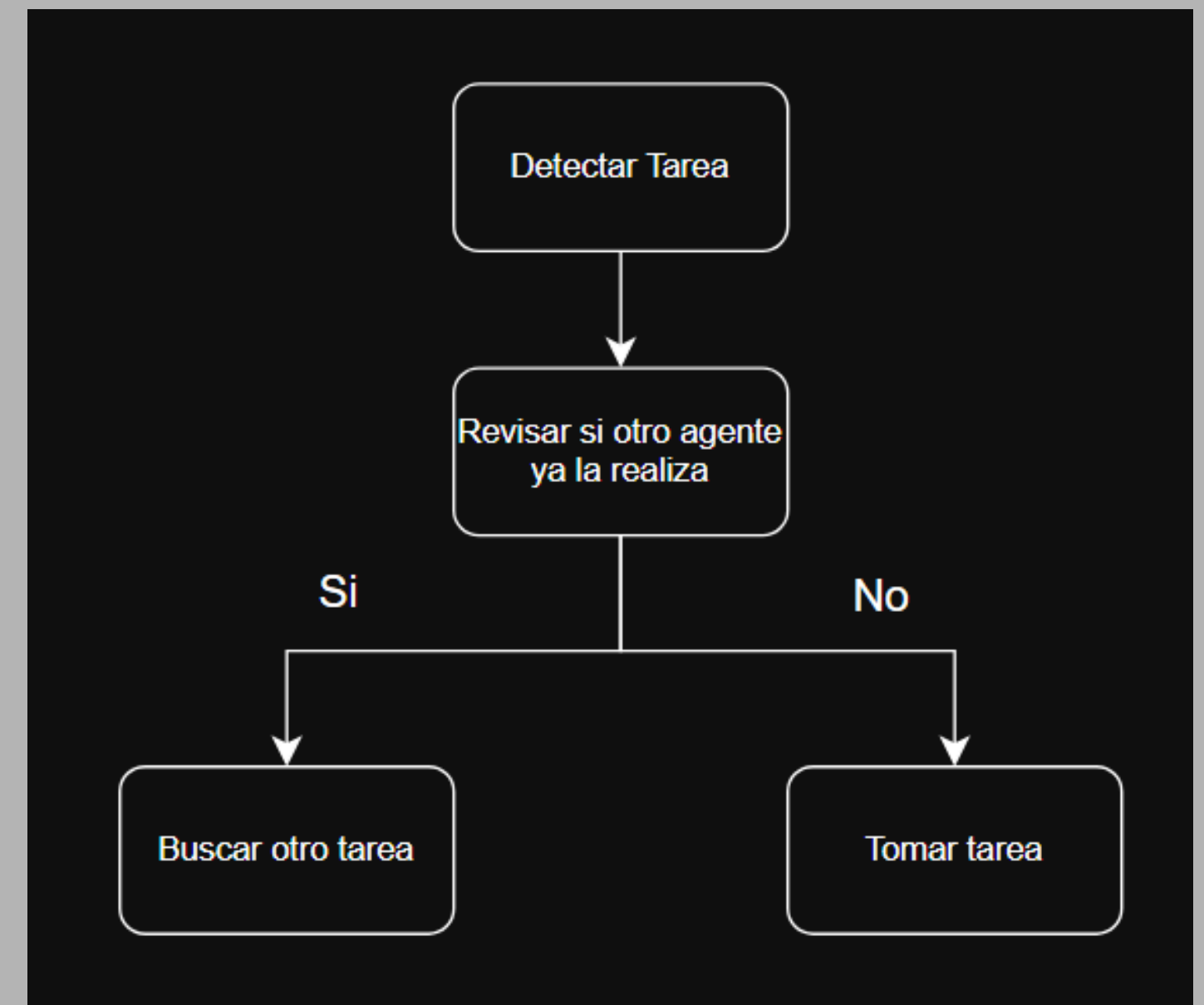
- | | |
|---------------------|------------------------|
| 1. Localizar POI | 4. Transportar víctima |
| 2. Alcanzar víctima | 5. Llegar al exterior |
| 3. Despejar ruta | |

Comportamientos de los agentes

Comportamiento de cooperación y coordinación

Planteamos que los seis agentes compartan objetivos para evitar trabajo duplicado y que no vayan todos al mismo punto.

Así los agentes podrán coordinar actividades para alcanzar metas cooperativamente.



Comportamientos de los agentes

Comportamiento adaptativo

Los agentes no mantendrán permanentemente una decisión inicial, sino que podrán reevaluar sus acciones conforme cambie el estado del incendio.



Propuestas de algoritmos

Para optimizar el movimiento de los agentes, implementaremos el algoritmo de búsqueda de rutas A*. Este método calcula la trayectoria más eficiente priorizando el ahorro de Puntos de Acción (AP) sobre la simple distancia física.

1. **Costo Real (G):** Gasto de AP acumulado en el trayecto (ej. caminar = 1 AP, romper muro = 5 AP).
2. **Heurística (H):** Distancia Manhattan aproximada hacia el objetivo, ignorando obstáculos.

Problemas y retos

- Sincronización entre Python y Unity: Mantener consistente el estado de la simulación entre la lógica desarrollada en Python y la visualización de Unity. Consolidar una buena comunicación entre Cliente-Servidor
- Lograr una actualización de mapa a los agentes cuando alguno de estos destruya una pared, para que los demás se enteren.
- Balance entre rescatar y extinguir: cuándo conviene ir directamente por una víctima y cuándo controlar primero el incendio.

Siguientes pasos:

- **Comportamiento de los agentes**
- **Unificación entre los scripts**
- **Creación de la interfaz en unity**
- **Creación de la API que conecte la simulación con Unity**